



Deep Neural Network

BITS Pilani

Pilani Campus

Deep Neural Network



Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Agenda



- Optimization Algorithms
 - Gradient Descent
 - Stochastic Gradient Descent
 - Mini batch Gradient Descent
 - Gradient Descent with momentum
 - ADAgrad
 - RMSProp
 - Adam

Optimization Algorithm



- Optimization algorithms train deep learning models.
- Optimization algorithms are the tools that allow
 - continue **updating model parameters**
 - to **minimize the value of the loss function**, as evaluated on the training set.
- In optimization, a loss function is often referred to as the **objective function** of the optimization problem.
- By tradition and convention most optimization algorithms are concerned with **minimization**.
- The goal of optimization is to reduce the training error. The goal of deep learning is to reduce the generalization error, this requires reduction in overfitting also.

Why Optimization Algorithm?



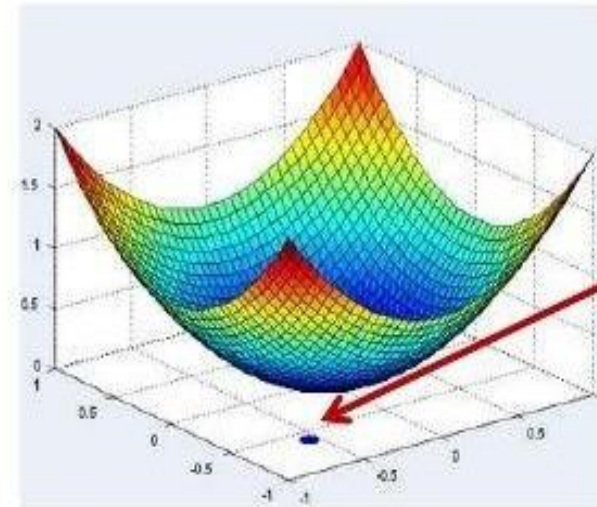
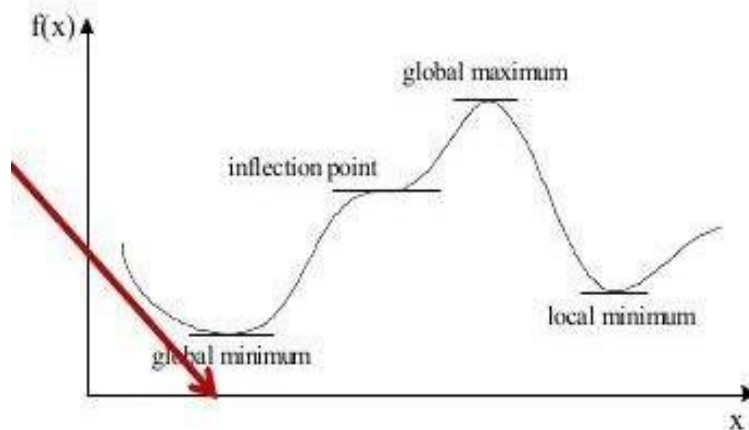
- Understanding the principles of different optimization algorithms and the role of their hyperparameters will enable us to tune the hyperparameters in a targeted manner to improve the performance of deep learning models

Optimization

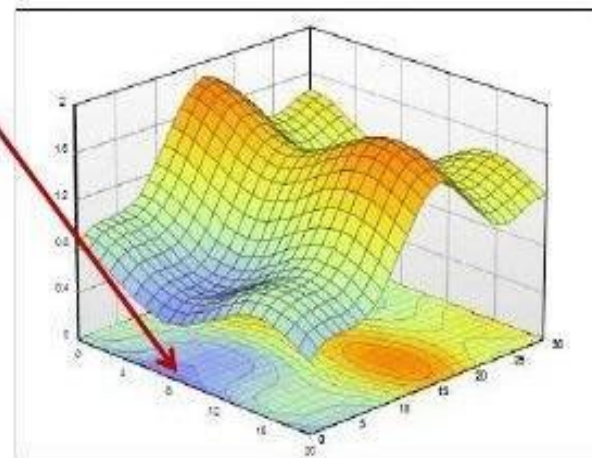
innovate

achieve

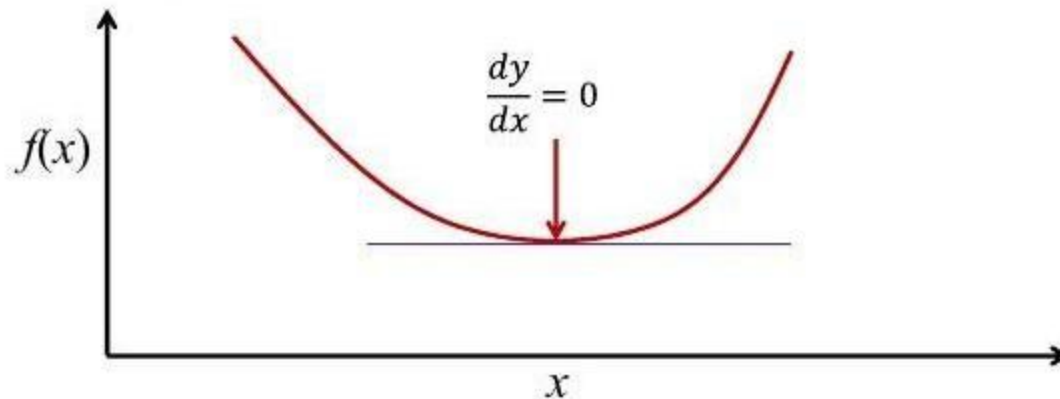
lead



- General problem of optimization: find the value of x where $f(x)$ is minimum



Finding Minimum of a Function



- Find the value x at which $f'(x) = 0$
 - Solve

$$\frac{df(x)}{dx} = 0$$

- The solution is a “turning point”
 - Derivatives go from positive to negative or vice versa at this point
- But is it a minimum?

Derivatives and critical points



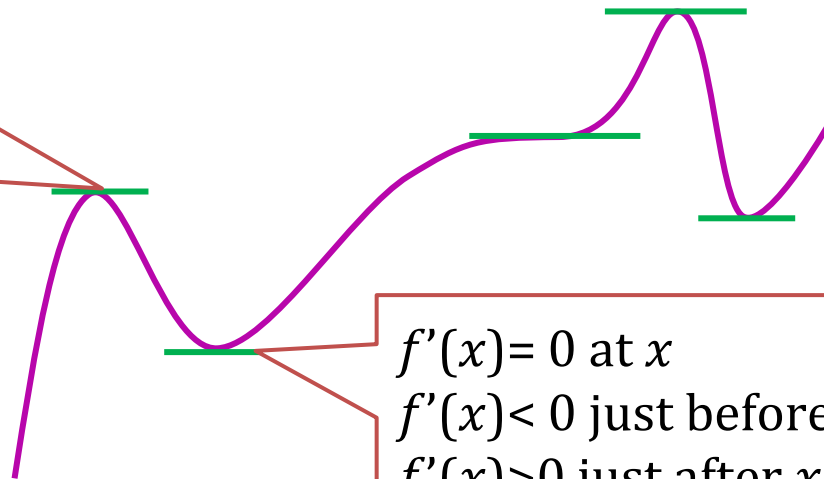
$f'(x) = 0$ at x
 $f'(x) = 0$ just before x $f'(x) = 0$ just after x
 x may be a saddle

Maxima: Imagine a hill where the slope flattens at the top. The second derivative tells us whether the slope changes in a downward direction.

Minima: Imagine a valley where the slope flattens at the bottom. The second derivative tells us whether the slope changes in an upward direction.

Saddle Point: Imagine a flat or pointy shape where the slope is flat, but the function has different curvatures in different directions. The second derivative is zero, and we cannot definitively classify it as a maximum or minimum.

$f'(x) = 0$ at x ,
 $f'(x) > 0$ just before x
 $f'(x) < 0$ just after x
 x is a maxima

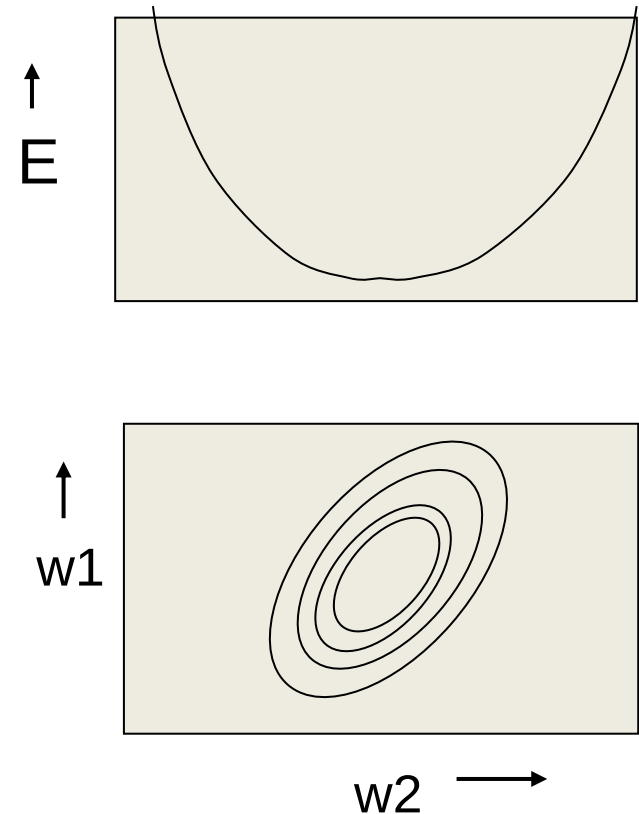


$f'(x) = 0$ at x
 $f'(x) < 0$ just before x
 $f'(x) > 0$ just after x
 x is a minima

$f'(x) = 0$ and $f''(x) < 0$, x is a maxima
 $f'(x) = 0$ and $f''(x) > 0$, x is a minima
 $f'(x) = 0$ and $f''(x) = 0$, x may be a saddle.

Reminder: The error surface for a linear neuron

- The error surface lies in a space with a horizontal axis for each weight and one vertical axis for the error.
 - For a linear neuron with a squared error, it is a quadratic bowl.
- For multi-layer, non-linear nets the error surface is much more complicated.
 - **But locally, a piece of a quadratic bowl is usually a very good approximation.**
 - **GD can be a good approximation**



Gradient Descent



One-Dimensional Gradient Descent

- In one-dimensional gradient descent, the algorithm adjusts a single parameter to minimize a function by following the negative direction of the gradient.

Multivariate Gradient Descent

- For multivariate functions, gradient descent adjusts multiple parameters simultaneously by iteratively updating them in the direction of the negative gradient.

Gradient descent



- Objective function is usually the average of the loss functions for each example in the training dataset.

n = no of examples training dataset

$f_i(\mathbf{x})$ = loss function with respect to the training example of index i

$\mathbf{x}$ = parameter vector

- objective function :

$$f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x}).$$

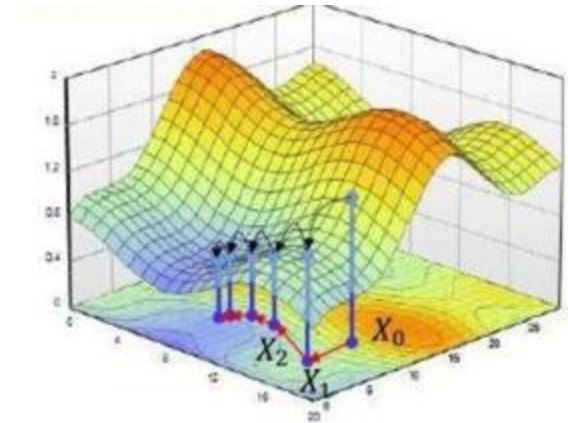
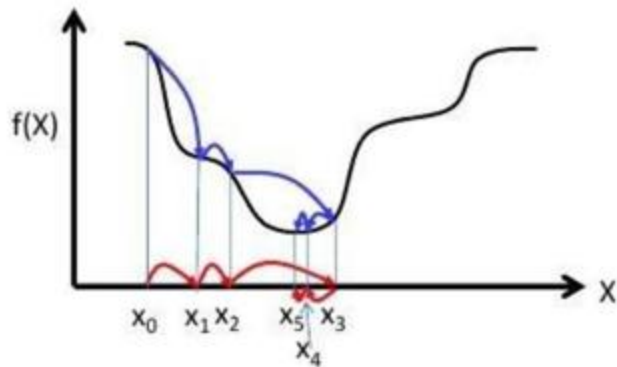
- gradient of the objective function at $\mathbf{x}$:

$$\nabla f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}).$$

- Update rule $\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f(\mathbf{x})$

- In vanilla GD, computational cost for each independent variable iteration is $O(n)$ which grows linearly with n
- Larger the training dataset is larger, higher the cost of GD for each iteration**

Gradient descent



- Iterative solutions
 - Start from an initial guess X_0 for the optimal X
 - Update the guess towards a (hopefully) “better” value of $f(X)$
 - Stop when $f(X)$ no longer decreases
- Problems:
 - Which direction to step in
 - How big must the steps be
- Loss functions of most deep learning models are non-convex
- Non-convex functions have multiple minima. Usually harder to optimize as compared to convex functions

Gradient descent

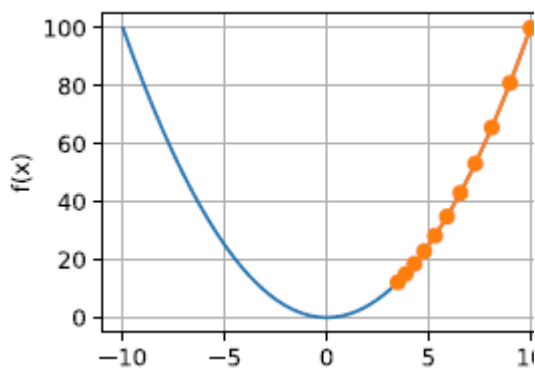


- Iterative solution:
 - Start at some point
 - Find direction in which to shift this point to decrease error
 - This can be found from the derivative of the function
 - A positive derivative $\rightarrow$ moving left decreases error
 - A negative derivative $\rightarrow$ moving right decreases error
 - Shift point in this direction

Learning rate

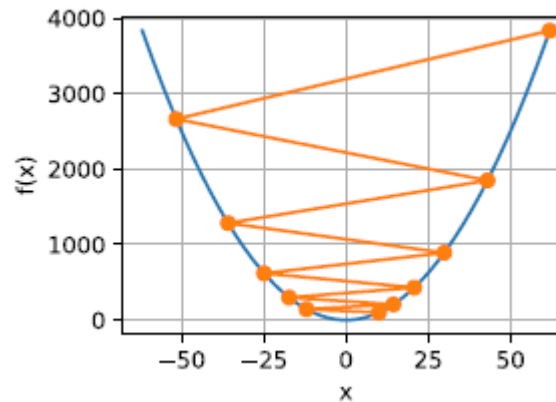


- The role of the learning rate is to moderate the degree to which weights are changed at each step.
- Learning rate η is set by the algorithm designer.



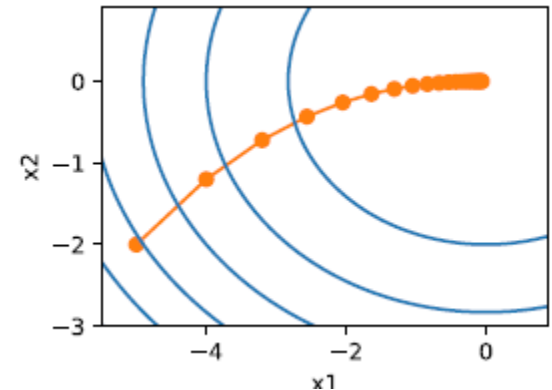
Slow Learning

If we pick η too small, we make little progress.



Oscillations

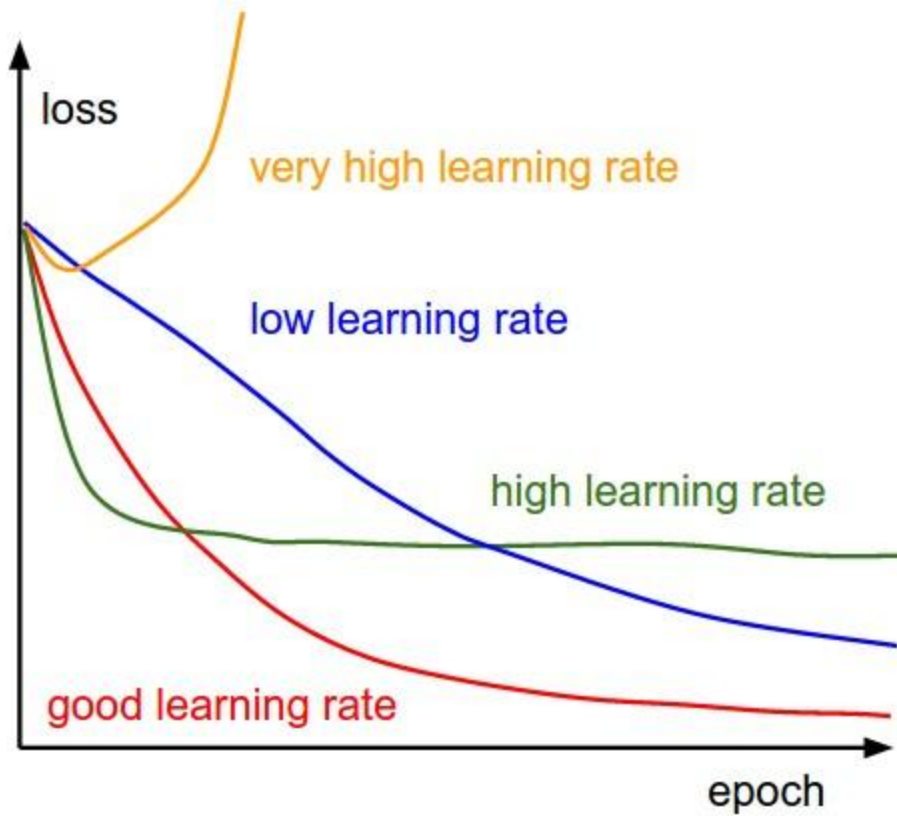
If we pick η too large, the solution oscillates and in the worst case it might diverge.



Optimal η

GD converges, a suitable η is often found only after multiple experiments

Learning rate



Local minima vs global minimum for DNN



- Tweak the learning rate
 - Gradually reduce the learning rate, then increase it and slowly reduce it, again, several times.
 - Increasing the learning rate reduces the stability of the algorithm, but gives the algorithm the ability to jump out of a local optimum
- Another takeaway
 - Finding the global minimum is probably **not the best** as it would probably represent extreme overfitting on the training set.
 - Empirical evidence shows that generalization performance is same for local minima solution and global minimum solution

Stochastic gradient descent

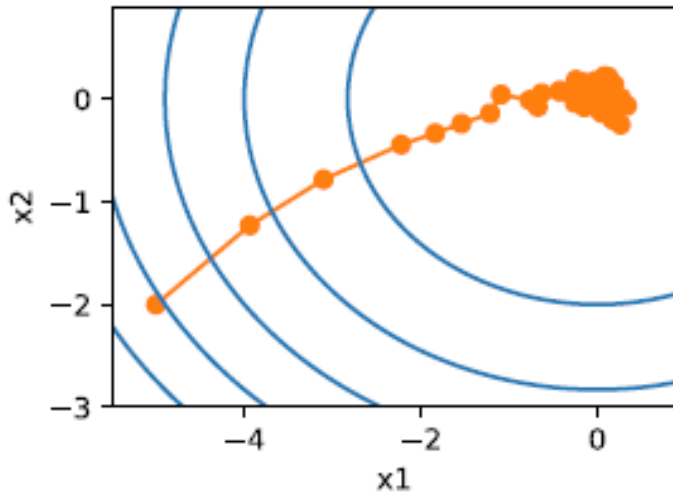


- SGD reduces computational cost at each iteration
- At each iteration of stochastic gradient descent, we uniformly sample an index $i \in \{1, \dots, n\}$ for data examples at random
- compute the gradient $\nabla f_i(\mathbf{x})$ to update $\mathbf{x}$:

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f_i(\mathbf{x}),$$

- computational cost for each iteration drops from $O(n)$ of the GD to the constant $O(1)$, hence **it is computationally efficient**

Stochastic gradient descent



- It may diverge near minimum
- This leaves us with the only alternative: change the learning rate η
- However, if we pick this too small, we will not make any meaningful progress initially
- On the other hand, if we pick it too large, we will not get a good solution
- The only way to resolve these conflicting goals is to reduce the learning rate *dynamically* as optimization progresses

Dynamic learning rate



- Replace η with time-dependent learning rate $\eta(t)$
 - adds to the complexity of controlling convergence of an optimization algorithm.

$$\eta(t) = \eta_i \text{ if } t_i \leq t \leq t_{i+1} \quad \text{piecewise constant}$$

$$\eta(t) = \eta_0 \cdot e^{-\lambda t} \quad \text{exponential decay}$$

$$\eta(t) = \eta_0 \cdot (\beta t + 1)^{-\alpha} \quad \text{polynomial decay}$$

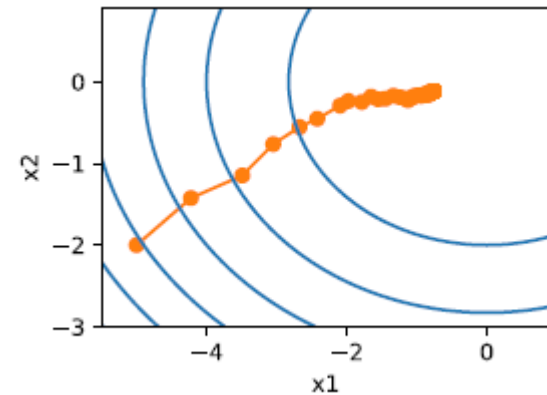
1. Piecewise constant
 - decrease the learning rate, e.g., whenever progress in optimization stalls.
 - This is a common strategy for training deep networks.
2. Exponential decay
 - Leads to premature stopping before the algorithm has converged.
3. Polynomial decay with $\alpha = 0.5$

Dynamic learning rate

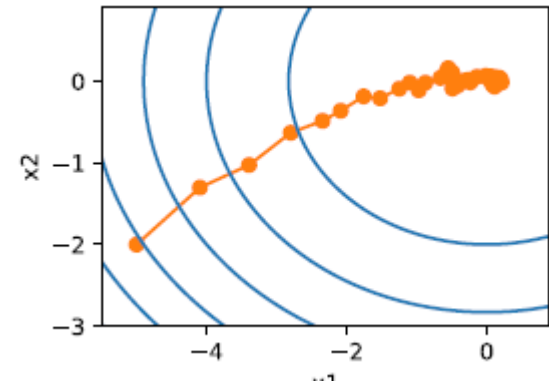


- Variance in the parameters is significantly reduced. But,
- The algorithm fails to converge at all.

Exponential decay



Polynomial decay



- Convergence gets better after only 50 steps.

Mini batch GD



- GD is not particularly *data efficient*
- SGD is not particularly *computationally efficient* since CPUs and GPUs cannot exploit the full power of vectorization
- Mini batch GD: read *mini batches* of data rather than single observations to update parameters
- Weight update:

$$\mathbf{g}_t = \partial_{\mathbf{w}} \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} f(\mathbf{x}_i, \mathbf{w})$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta_t \mathbf{g}_t$$

- Pick a mini batch (which is hyper parameter) that is large enough to offer good computational efficiency while still fitting into the memory of a GPU.
- For small training sets (e.g., <2000), use GD
- For larger sets, sizes between 64 to 512 (preferably powers of 2) are typical.

Steps for mini batch GD

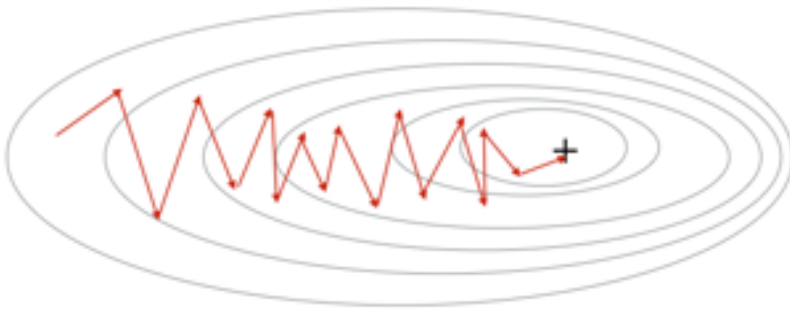


1. Split the data into mini-batches, e.g., $X^{(1)}$ through $X^{(1000)}$, $X^{(1001)}$ through $X^{(2000)}$, and so on.
2. For each mini-batch, perform forward propagation using only the data in that mini-batch.
3. Compute the cost function for that mini-batch.
4. Implement backpropagation to compute gradients.
5. Update the weights and biases using the gradients.
 1. Enables progress in gradient descent even when only partially through the training set.
 2. With GD, cost should decrease every iteration.
 3. For mini-batch GD, cost might not decrease every iteration due to different training batches.
 4. Cost function J should generally trend downwards but may oscillate due to varying difficulty of mini-batches.

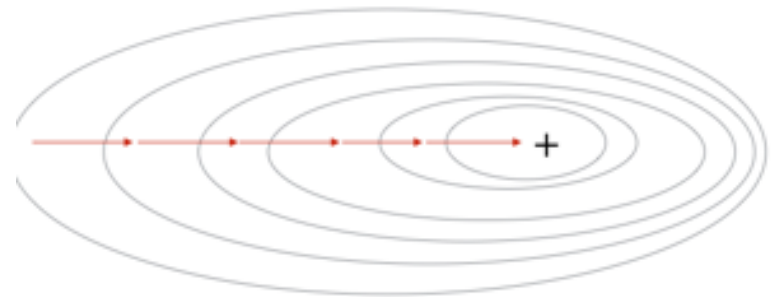
GD vs SGD vs Mini batch GD



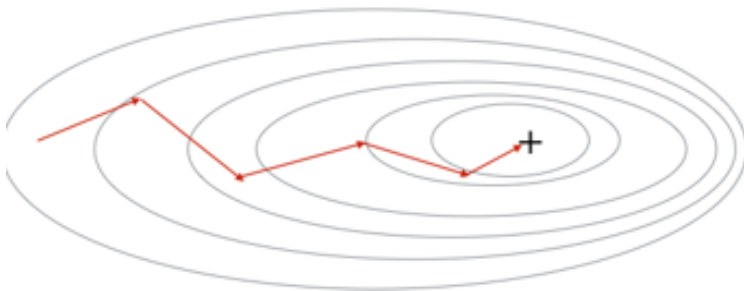
Stochastic gradient descent
Noisy, oscillates around the minimum
but may not converge to it.



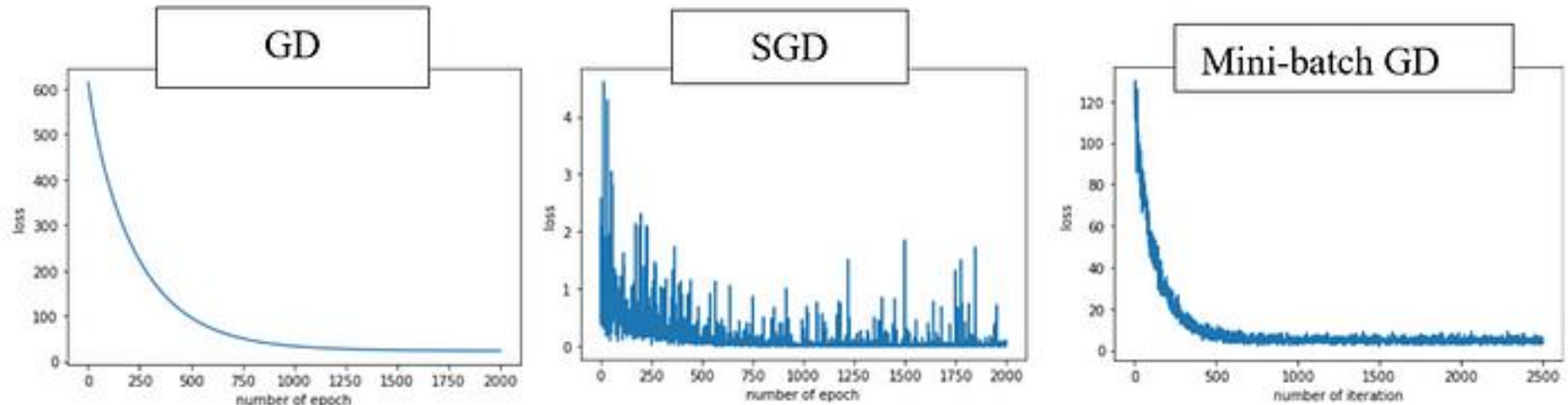
Gradient descent
Takes consistent steps toward minimum.



Mini batch gradient descent
Noisy, oscillates around the minimum but may not converge to it.



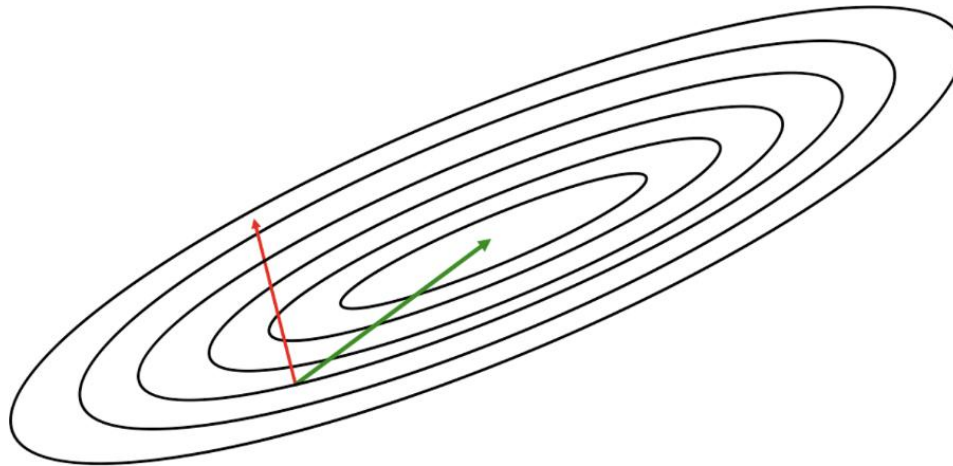
GD vs SGD vs Mini batch GD



GD is computationally heavy, converges to a “flat minima” but performs well on the test data.

Drawback of gradient based methods

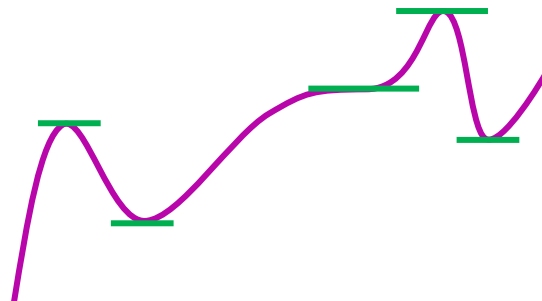
- The most critical challenge to optimizing deep networks is finding the correct trajectory to move in.
- Gradient isn't usually a very good indicator of the good trajectory.
- when the contours are perfectly circular , gradient always point in the direction of the local minimum.
- However, if the contours are extremely elliptical (as is usually the case for the error surfaces of deep networks), the gradient can be as inaccurate as 90 degrees away from the correct direction!



Error surface of DNN



- A critical point can only be a local minimum if it appears as a local minimum in every single one of the d one-dimensional subspaces
- As the dimensionality of our parameter space increases, local minima become exponentially more rare
- As our function has more and more dimensions (i.e., we have more and more parameters in our model), saddle points are exponentially more likely than local minima



Exponentially Weighted Averages (EWA)



- Technique used to calculate the moving average of a time series, where more recent data points are given higher weights, and older data points are given lower weights.
- The weights decline exponentially as the data points get older, hence the name “exponentially weighted”
- Applied in various optimization algorithms in deep learning, such as Gradient Descent with Momentum, RMSprop, and Adam
- The EWA is calculated using the following formula:

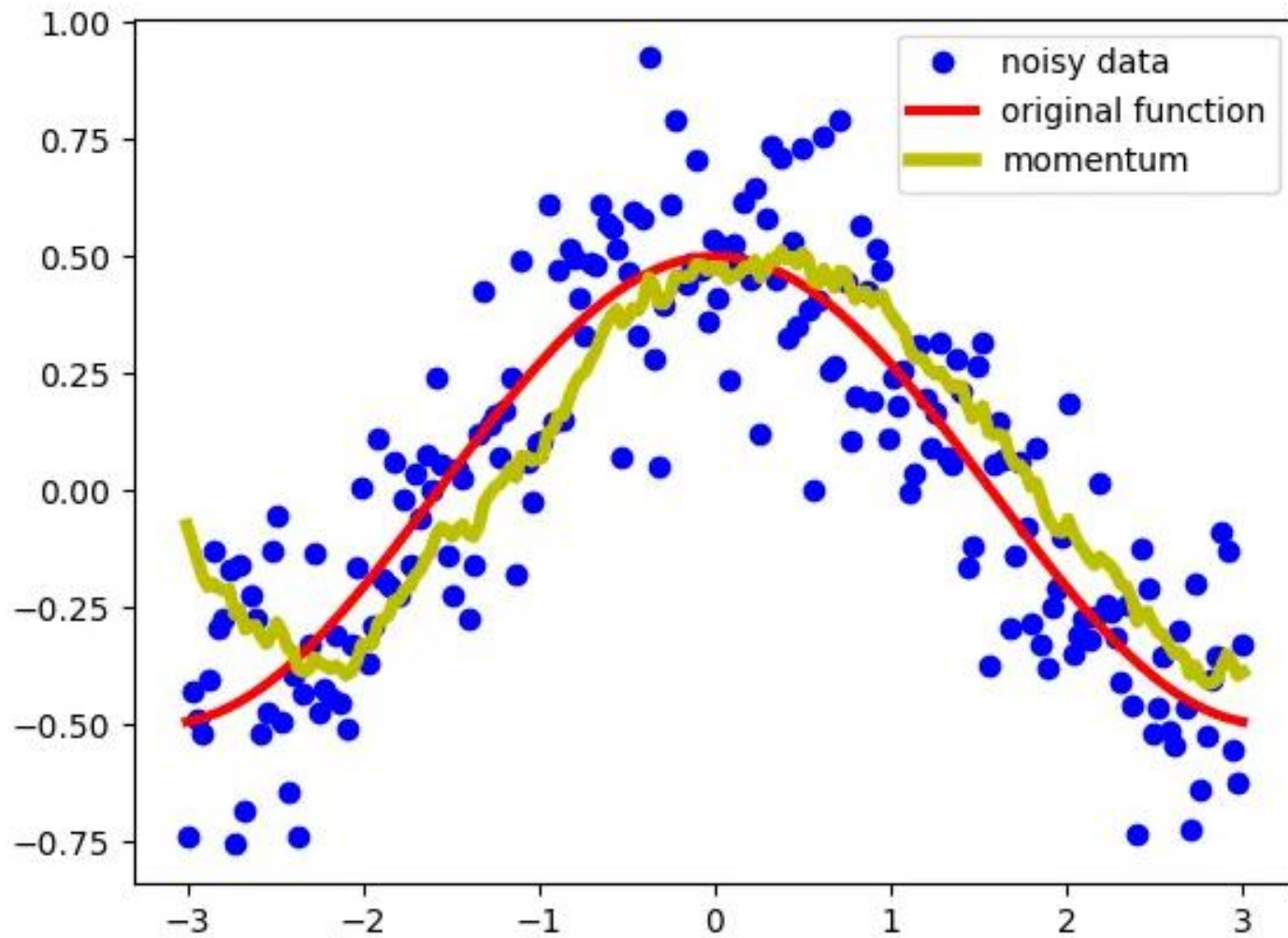
$$V_t = \beta * (V_{t-1}) + (1 - \beta) * \text{NewSample}$$

V_t = weighted average at time t

β = coefficient of momentum, parameter that determines weight given to previous values (between 0 and 1)

NewSample is the new data point at time t

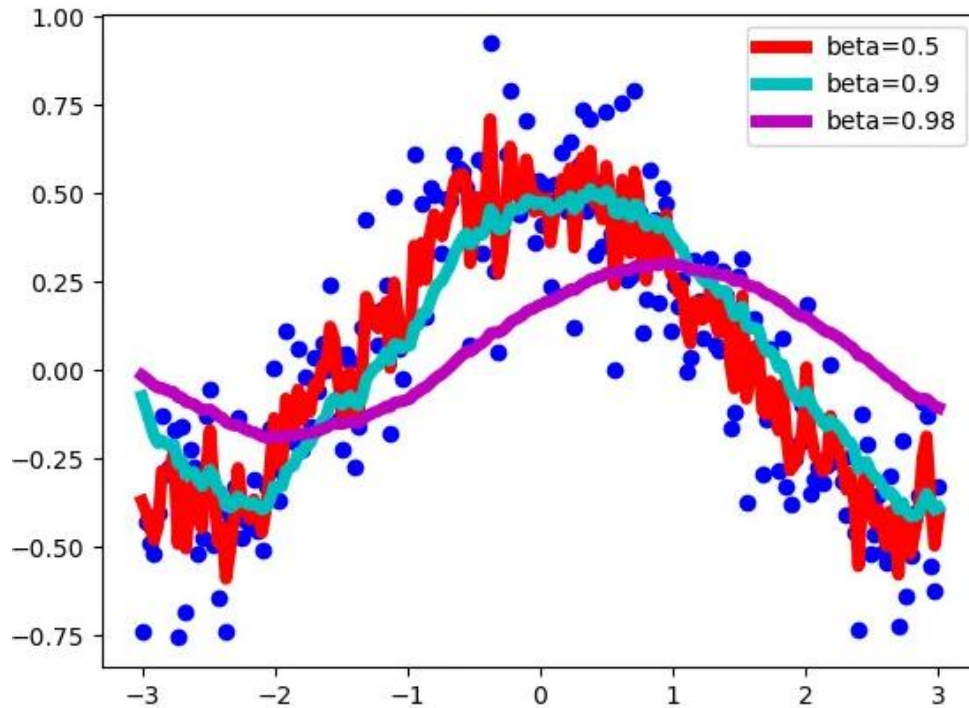
EWA



EWA



$$V_t = \beta * (V_{t-1}) + (1-\beta) * \text{NewSample}$$



$$n_{\beta} = \frac{1}{1 - \beta}$$

n_{β} as the numbers of observations used to adapt your EWA.

β	n_{β}	EWA
0.9	10	Adapts normal
0.98	50	Adapts slowly
0.5	2	Adapts quickly

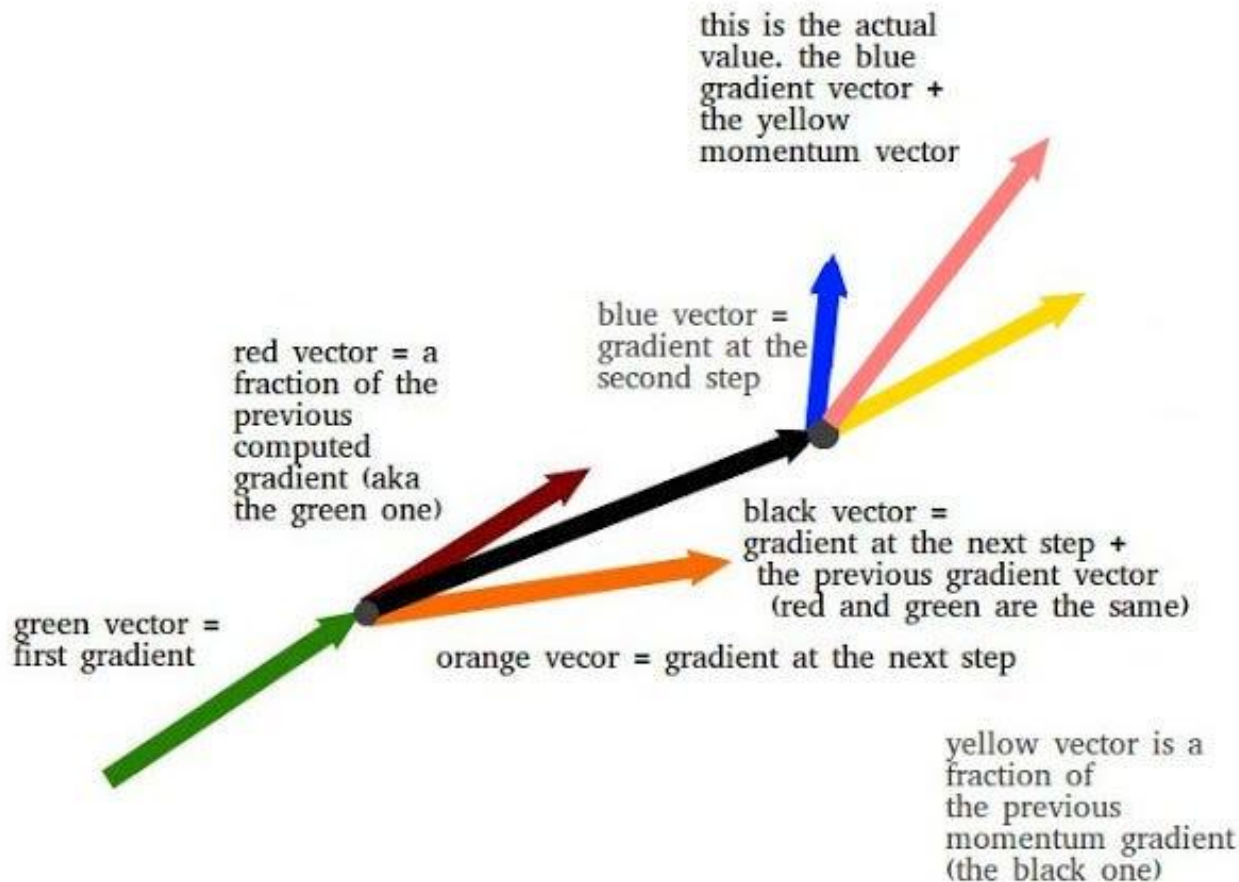
Gradient Descent with Momentum

$$V_t = \beta * (V_{t-1}) + (1-\beta) * \text{NewSample}$$

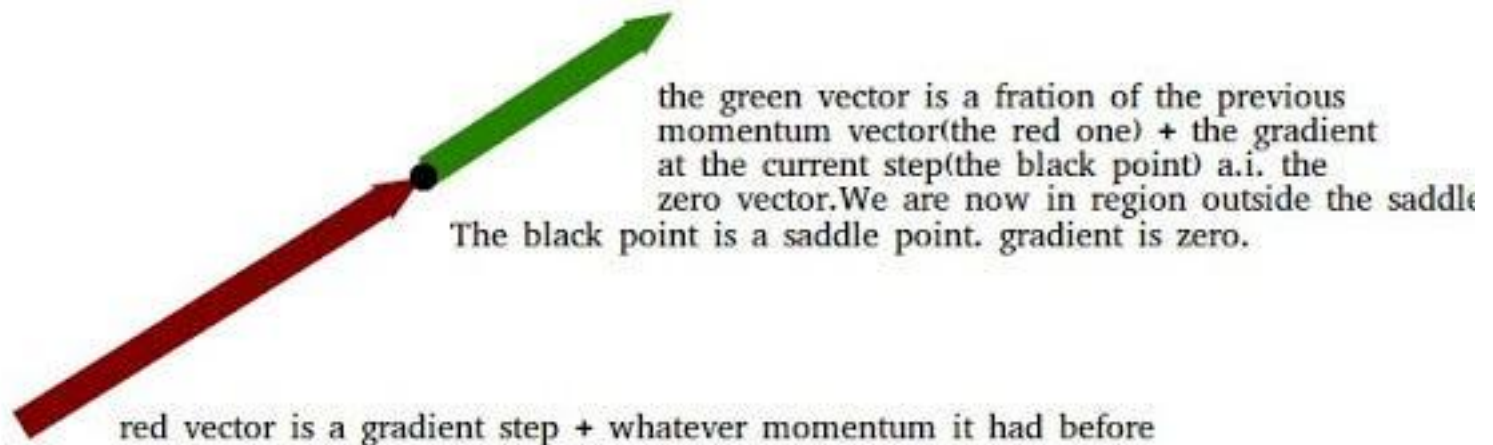
- Added a mechanism in GD for **aggregating a history of past gradients to accelerate convergence**
- Taking knowledge from previous steps about where we should be heading. We are introducing a new hyperparameter β
- The term v_t is called momentum
- This term accumulates the gradients from the past several steps
- This method updates the parameters w in the direction of the weighted average of the past gradients

$$\begin{aligned} v_t &\leftarrow \beta v_{t-1} + g_{t,t-1} \\ w_t &\leftarrow w_{t-1} - \eta v_t \end{aligned}$$

Gradient Descent with Momentum



Gradient Descent with Momentum



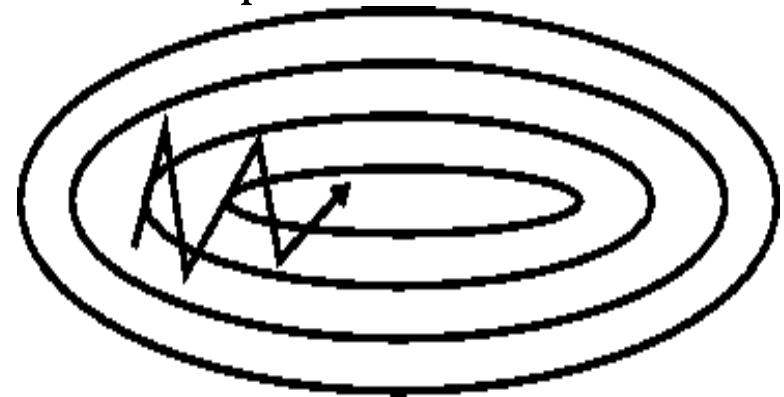
Momentum Update



Plain gradient



update With momentum



- Momentum averages out the oscillations
- In a vertical direction, oscillations are reduced because positive and negative numbers average out, Reduces updates for dimensions whose gradients change directions
- In a horizontal direction, the movement remains aggressive, which speeds up the descent, Increases updates for dimensions whose gradients point in the same directions.

Gradient Descent with Momentum



Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α .

Require: Initial parameter θ , initial velocity v .

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$

 Apply update: $\theta \leftarrow \theta + \mathbf{v}$

end while

$$v_t \leftarrow \beta v_{t-1} + g_{t,t-1}$$

$$w_t \leftarrow w_{t-1} - \eta v_t$$

Gradient Descent with Momentum



- Momentum replaces gradients with a leaky average over past gradients. This accelerates convergence significantly.
- Momentum prevents stalling of the optimization process that is much more likely to occur for stochastic gradient descent.
- The effective number of gradients is given by $1 / (1 - \beta)$ due to exponentiated downweighting of past data.
- Implementation is quite straightforward but it requires us to store an additional state vector (momentum v).

Gradient Descent with Momentum



Gradient descent with momentum uses the momentum of the gradient for parameter optimization

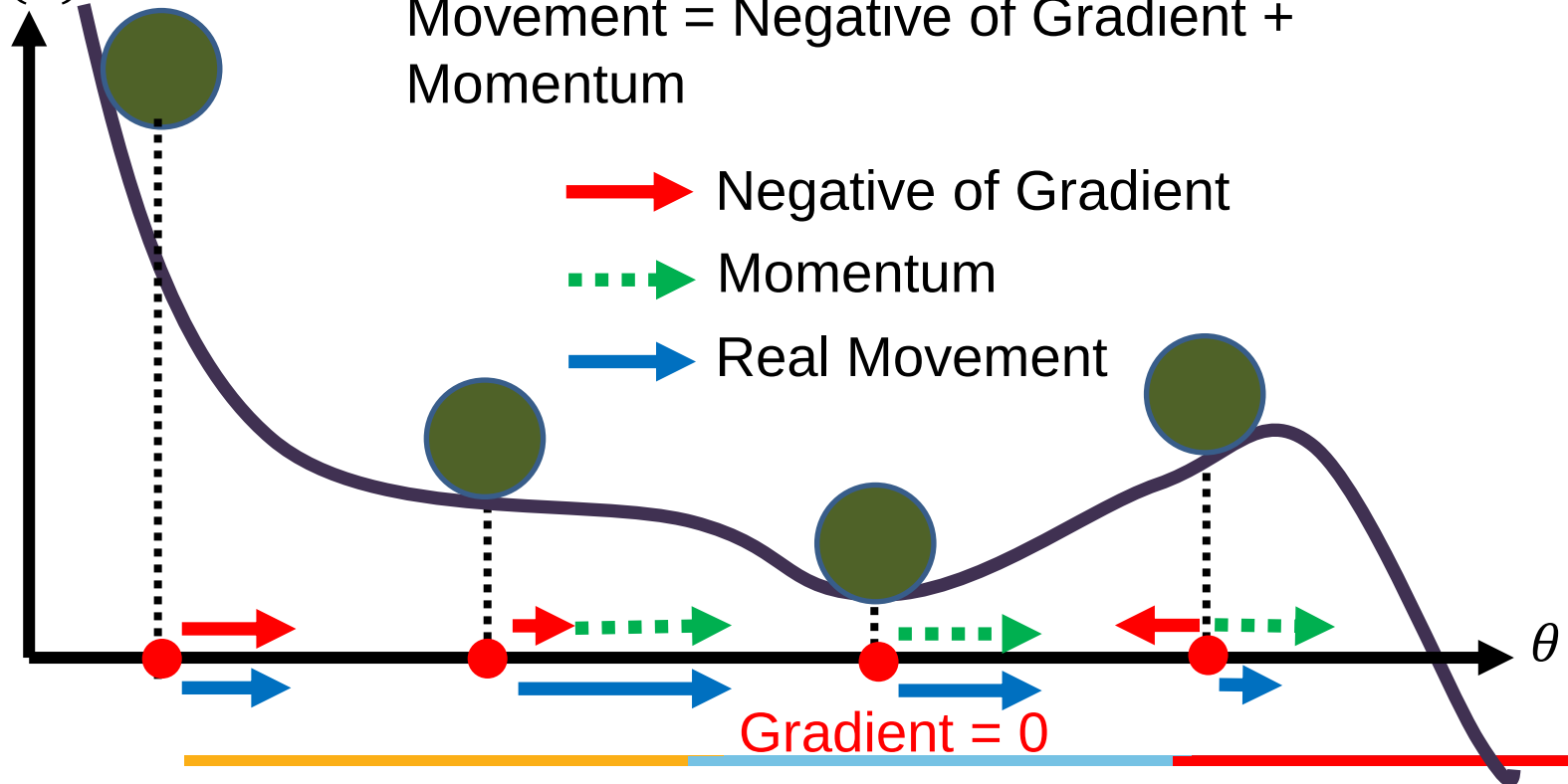
cost $\mathcal{L}(\theta)$

Movement = Negative of Gradient + Momentum

→ Negative of Gradient

→ Momentum

→ Real Movement



Solved problem SGD and SGD + momentum



Consider the following loss function, with initial value of $W^{\{0\}} = -2.8$, and learning rate $\eta = 0.05$ and $\beta=0.7$. Use SGD and SGD + momentum to find the updated value of $W^{\{1\}}$ after the first iteration.

$$L(w) = 0.3 * w^4 - 0.1 * w^3 - 2 * w^2 - 0.8 * w$$

Answer

Iteration	g_i	w_i
1	-18.2943	-1.88527
2	-2.36614	-1.76697

Iteration	g_i	v_i	w_i
1	-18.294	0	-1.88527
2	-2.3661	-15.172	-1.126



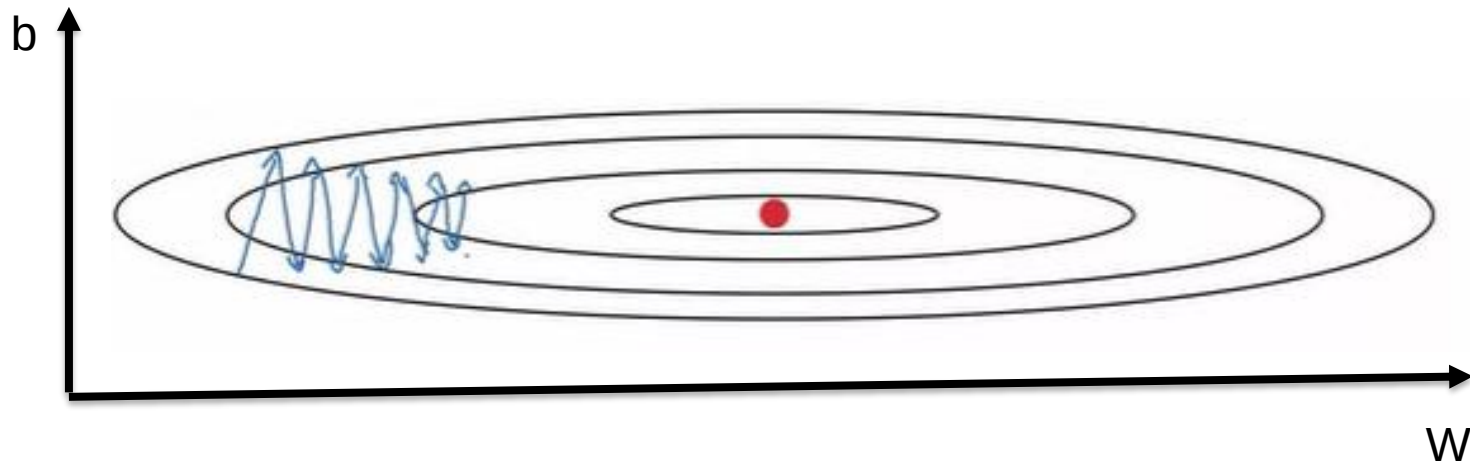
Adagrad - motivation



- In GD same LR for all parameters and iteration.
- What happens to features that occur infrequently (sparse features) e.g w is sparse, b is not (as x_0 is always 1) e.g:

x	3.5	0	0	0	0	-0.5
y	0.5	0.5	0.5	0.45	0.48	0.47

- Parameters associated with infrequent features only receive meaningful updates whenever these features occur



Adagrad



- Decay the learning rate for parameters in proportion to their update history
- An individual learning rate per parameter(feature)
- Accumulating past squared gradients in $\mathbf{s}_t$

$$\mathbf{g}_t = \partial_{\mathbf{w}} l(y_t, f(\mathbf{x}_t, \mathbf{w})),$$

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{g}_t^2, \quad \leftarrow \text{Accumulate the history}$$

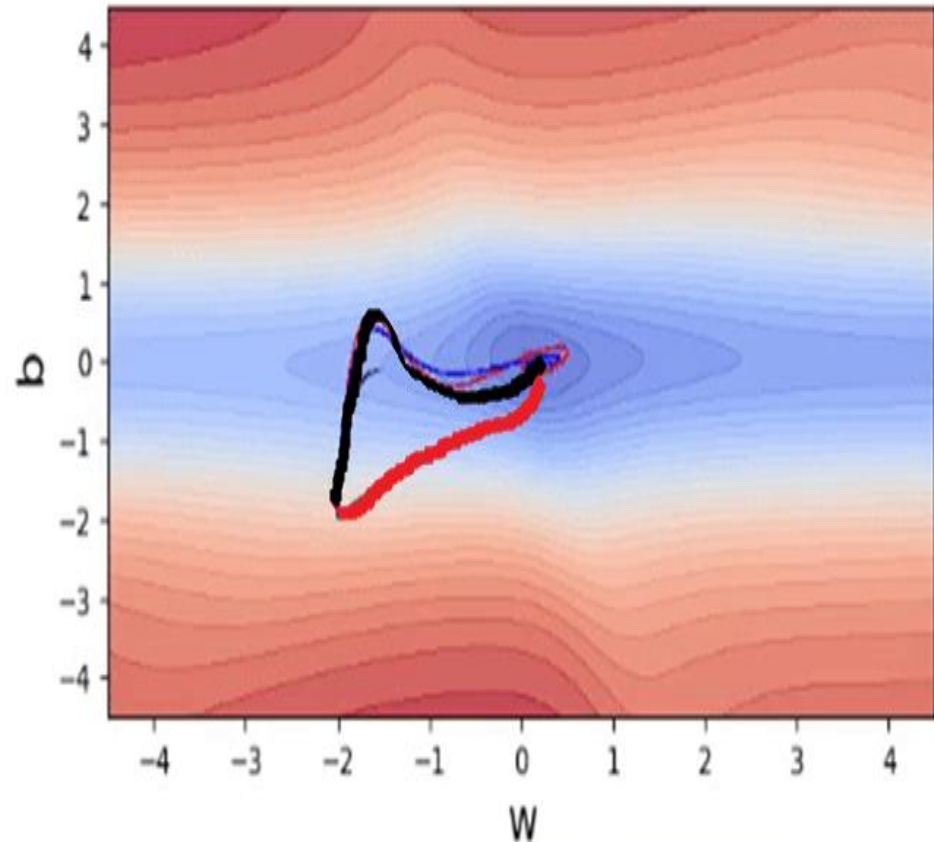
$$\mathbf{w}_t = \mathbf{w}_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \cdot \mathbf{g}_t.$$

- ϵ is a smoothing term that avoids division by zero
- Initialize $\mathbf{s}_0 = 0$
- Adagrad's main benefits is that it eliminates the need to manually tune the learning rate.
- Most implementations use a default value of 0.01 and leave it at that
- Application: In natural language processing and image recognition applications

Adagrad



- It adapts the learning rate to the parameters
 - smaller updates (i.e. low learning rates) for parameters associated with frequently occurring features
 - larger updates (i.e. high learning rates) for parameters associated with infrequent features



Drawback of AdaGrad



- AdaGrad decays the learning rate very aggressively (as the denominator grows).
- As a result, after a while, the frequent parameters will start receiving very small updates because of the decayed learning rate.

$$\begin{aligned} s_t &= s_{t-1} + g_t^2, \\ w_t &= w_{t-1} - \frac{\eta}{\sqrt{s_t} + \epsilon} \cdot g_t \end{aligned}$$

- As a result, the algorithm may experience sluggish convergence or even premature stalling
- To avoid this why not decay the denominator and prevent its rapid growth

RMSProp



- The issue :
 - Adagrad accumulates the squares of the gradient $\mathbf{g}_t$ into a state vector $s_t = s_{t-1} + g_t^2$
 - As a result $\mathbf{s}_t$ keeps on growing without bound due to the lack of normalization

$$s_t = s_{t-1} + g_t^2,$$
$$\mathbf{w}_t = \mathbf{w}_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \cdot \mathbf{g}_t$$

- Use a leaky average in the same way we used in the momentum method

$$s_t \leftarrow \gamma s_{t-1} + (1 - \gamma) g_t^2,$$
$$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot \mathbf{g}_t.$$

Parameter $\gamma > 0$

The constant $\epsilon > 0$ is typically set to 10^{-6}

- **Faster convergence compared to AdaGrad**
- works well on big and redundant datasets

Review of techniques learned so far



1. Stochastic gradient descent
 - more compute effective than Gradient Descent when solving optimization problems
 - Mini batch Stochastic gradient descent affords significant additional efficiency arising from vectorization, using larger sets of observations in one mini batch.
 - This is the key to efficient multi-machine, multi-GPU and overall parallel processing.
2. Momentum
 - added a mechanism for aggregating a history of past gradients to accelerate convergence.
3. Adagrad
 - used per-coordinate scaling to allow for a computationally efficient preconditioner.
4. RMSProp
 - leaky average + dynamic learning rate

Adam-Adaptive Moment Estimation

- Adam combines all these techniques into one efficient learning algorithm.
- *computes individual adaptive learning rates for different parameters from estimates of the first and second moments of the gradients.*
- utilize the momentum concept from “SGD with momentum” and adaptive learning rate from “RMSProp”
- Adam can diverge due to poor variance control.
(disadvantage)

$$\begin{aligned}\mathbf{v}_t &\leftarrow \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \\ \mathbf{s}_t &\leftarrow \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2.\end{aligned}$$

← V for Momentum

← S for RMSProp

Adam Algorithm

$$\mathbf{v}_t \leftarrow \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1) \mathbf{g}_t,$$

← first moment (the mean) of gradient

$$\mathbf{s}_t \leftarrow \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2.$$

← second moment (uncentered variance) of gradients

- β_1 and β_2 are nonnegative weighting parameters.
- Common choices for them are $\beta_1 = 0.9$ and $\beta_2 = 0.999$.
- Initialize $\mathbf{v}_0 = \mathbf{s}_0 = 0$
- Normalize the state variables (bias correction) →

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_1^t} \text{ and } \hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2^t}.$$

Bias-correction helps Adam slightly outperform RMSprop

- Rescale the gradient →

$$\mathbf{g}'_t = \frac{\eta \hat{\mathbf{v}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon}.$$

- Compute updates →

$$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \mathbf{g}'_t.$$

Optimization algorithm comparison



Gradient Descent

$$w_t \leftarrow w_{t-1} - \eta g_t$$

GD with Momentum: replaces gradients with a leaky average over past gradients

$$v_t \leftarrow \beta v_{t-1} + g_{t,t-1}$$

$$w_t \leftarrow w_{t-1} - \eta v_t$$

AdaGrad (Adaptive learning rate): An individual learning rate per parameter

$$s_t = s_{t-1} + g_t^2,$$

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \cdot g_t$$

RMSProp : exponentially decaying average of past squared gradients

$$s_t \leftarrow \gamma s_{t-1} + (1 - \gamma) g_t^2,$$

$$x_t \leftarrow x_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t.$$

Adam : exponentially decaying average of past gradients + exponentially decaying average of past squared gradients

$$\mathbf{v}_t \leftarrow \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1) \mathbf{g}_t,$$

$$\mathbf{s}_t \leftarrow \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2.$$

ADAM VS SGD



Adaptive methods (such as Adam) do not generalize as well as SGD with momentum when tested on a diverse set of deep learning tasks

*In the research paper by Nitish Shirish Keskar, Richard Socher :
Improving Generalization Performance by Switching from Adam to
SGD :*

Earlier stages of training Adam still outperforms SGD but later the learning saturates. They proposed simple strategy in which they start training deep neural network with Adam but then switch to SGD when certain criteria hits

Optimization algorithm comparison



Variant	Advantages	Disadvantages
Batch gradient descent	Guaranteed convergence to global optimum	Computationally expensive for large datasets, slow convergence
Stochastic gradient descent	Faster convergence, more efficient for large datasets	High variance, may not converge to global optimum
Mini-batch gradient descent	Balanced convergence speed and computational cost, efficient for large datasets	Choice of mini-batch size can be a challenge
Momentum gradient descent	Faster convergence, less likely to get stuck in local minima	May overshoot and oscillate around the optimum
Adagrad	Adaptive learning rate, efficient for sparse data	Can stop learning too early
RMSProp	Adaptive learning rate, efficient for non-stationary problems	Can stop learning too early, requires tuning of hyperparameters
Adam	Adaptive learning rate, efficient for large datasets and noisy data	Can converge to suboptimal solutions, requires tuning of hyperparameters

References



Chapter – 12 Dive into Deep Learning

<https://d2l.ai/>



Thank You All !